Google Play — your checklist

You drive the console. I built the package. Work top to bottom; nothing here depends on a later step.

Everything in this file is free unless a price is stated. The only money is the $25 Play account fee, which you have already paid.

■ 17 AUG 2026 — THE RELEASE BUILD (do this one now; everything below it is done history)

Google granted production access on 15 Aug. Before pressing the final rollout button, one more build is needed: the test version of the app shell had **payments switched off** — people could install, but nobody could pay. The fix is already in the settings file; turning it into the app file (`.aab`) happens on your computer, because your signing key lives only there.

The two Google deadlines, corrected 17 Aug — Jacques's console is the truth, not the repo's settings file. The `.aab` uploaded for closed testing was built with older tools, so Google flagged it and offered an extension:

- **31 Aug (Android 16): ~~take the extension~~ — DONE. Jacques ALREADY REQUESTED IT (confirmed by him 17 Aug 2026). Deadline is now 1 Nov. DO NOT ask him to do this again.** This build then clears it for real: the update step pulls Google's current tools, which target Android 16.
- **29 Sep (Play Billing 8):** no extension exists, but this build clears it — the update step pulls the current billing library automatically.

■ FOUND 17 Aug 2026 — the key and the build workshop live at `C:\dayone`

`C:\dayone\app-claude-vibe-code-uwxxlk\TurnSomeDayIntoOneday\twa\` holds `android-upload.keystore` (made 27 Jul), the closed-testing `app-release-bundle.aab`, and the whole gradle project. The copy under Documents is a second download with no key in it — do not be fooled by it again. The USB "play key" note is a path to this folder; the password hunt continues in the "google key" note. **The upload-key reset was never needed and was not submitted.**

1. Find your build folder

The same folder from the first build — it contains `twa-manifest.json` and `android-upload.keystore`. If you can't remember where, search your PC in File Explorer for **android-upload.keystore**.

2. Put the new settings file in it

The repo is public, so you can download it straight from GitHub. Open:

`https://raw.githubusercontent.com/Jacqueslm/app/main/TurnSomeDayIntoOneday/twa/twa-manifest.json`

First rename the old file in your folder to `twa-manifest-old.json` (safety copy). Then save the page as `twa-manifest.json` into the folder (Ctrl+S — make sure the name is exactly that, not `.txt` on the end).

Or by hand: open your `twa-manifest.json` in Notepad and make these say: `"appVersionName": "1.0.1"` · `"appVersionCode": 2` · `"appVersion": "1.0.1"` · `"features": { "playBilling": { "enabled": true } }` · `"alphaDependencies": { "enabled": true }` — touch nothing else.

3. Open a command window in that folder

Click the folder's address bar in File Explorer, type `cmd`, press Enter.

4. Build — three lines, in this order

```
npm install -g @bubblewrap/cli
bubblewrap update
bubblewrap build
```

The first line refreshes the builder to Google's current tools — that is what guarantees the Play Billing 8.0 library and Android 16. The second rebuilds the project from the new settings. The third makes the app file and asks for your keystore password — the one you wrote down when you made the key. Out comes a fresh `app-release-bundle.aab`.

5. Upload — but do NOT roll out yet

Play Console → **Test and release** → **Production** → **Create new release** → upload the new `.aab` (it should show version 1.0.1, code 2). Release notes: "First public release." Press **Save** and stop. The rollout button waits until step 8 — Google won't let you create the payment products until it has seen a build that contains billing, which is why the order matters.

6. Create the three prices

Play Console → **Monetize**:

- **Subscriptions** → create ID `pro_monthly` — $9.99/month — add a **7-day free trial** offer. Activate.
- **Subscriptions** → create ID `pro_yearly` — $59.99/year. Activate.
- **In-app products** → create ID `pro_lifetime` — $149.99 one-time. Activate.

The IDs must be exactly those, letter for letter — they're what the app asks Google for.

7. The payment-verification key (do this WITH the AI)

The server never takes a phone's word that a purchase happened — it asks Google directly. For that, Railway needs one variable: `PLAY_SERVICE_ACCOUNT_JSON`. Setting it up is a 15-minute click-path through Google Cloud and Play Console. **When you reach this step, open a session and say "walk me through the payment service account"** — do it together, not from memory. Without this step, a person can pay Google and Pro won't unlock.

8. Roll out

Back to the saved release → **Review release** → **Start rollout to Production.** Managed publishing is off, so it goes live when Google approves. That click is the launch — it's yours.

Before anything else — two deadlines

- **31 August 2026** — new apps must target Android 16 (API 36). The project I generated already targets 36. An extension to 1 November 2026 can be requested if you miss it.
- The **12 testers / 14 days** requirement below is the long pole. It takes a minimum of two weeks of real calendar time. Start it before you polish anything else.

STEP 1 — Install the build tools (one time, on your own machine)

I could not build the `.aab` for you: this session's network policy blocks `dl.google.com`, so the Android SDK cannot be downloaded here. These commands do it on your machine. **Free.**

1. Install **Java JDK 17 or newer** — https://adoptium.net — free.
2. Install **Node.js 18+** if you do not have it — https://nodejs.org — free.
3. Open a terminal and run:

```
npm install -g @bubblewrap/cli
```

The first `bubblewrap` command will offer to download the Android SDK and JDK for you. Say yes. It is a few hundred MB and it is free.

STEP 2 — Generate your upload key (one time)

Read this first: with Play App Signing, Google holds the real app signing key. What you generate here is an **upload key**. If you lose it, Google can reset it — this is recoverable, unlike the old APK world. Back it up anyway.

From inside the `TurnSomeDayIntoOneday/twa/` folder:

```
keytool -genkeypair -v \
  -keystore android-upload.keystore \
  -alias upload \
  -keyalg RSA -keysize 2048 -validity 10000
```

It asks for a password twice and for your name and location. Use a real password and **write it down somewhere you will still have in five years** — a password manager, not a sticky note.

Back it up now, before you go further

Copy `android-upload.keystore` to **two places that are not this computer**:

- a password manager that stores files (1Password, Bitwarden), and
- an encrypted USB stick or a private cloud folder.

Store the password with it, but not in the same file.

It must never reach GitHub

I have already added the guard. Confirm it is working:

```
git check-ignore -v twa/android-upload.keystore
```

That must print a line naming `.gitignore`. If it prints nothing, **stop** and tell me — the key is about to be committed to a public history.

STEP 3 — Build the .aab

From the `twa/` folder:

```
bubblewrap init --manifest=https://www.turnsomedayintodayone.com/manifest.json
```

When it asks, accept the values already in `twa-manifest.json` — package `com.turnsomedayintodayone.app`, launcher name `Day One`, target SDK 36. Then:

```
bubblewrap build
```

You get `app-release-bundle.aab`. That is the file you upload.

STEP 4 — Create the app in Play Console

1. Play Console → **Create app**.
2. App name: **Turn Someday Into Day One**
3. Default language: **English (United States)**
4. App or game: **App**
5. Free or paid: **Free** (The Android build ships free-tier only. Pro is sold on the web.)

6. Tick the declarations, then **Create app**.

STEP 5 — Upload once to get your fingerprint

This is the step everyone gets stuck on, so read the order carefully.

The `assetlinks.json` file on your website needs a SHA-256 fingerprint that **does not exist yet**. It is created when Google signs your first upload. So:

1. Play Console → **Testing** → **Closed testing** → **Create new release**.
2. Upload `app-release-bundle.aab`.
3. Accept **Play App Signing** when prompted.
4. Go to **Setup** → **App signing**.
5. Copy the **SHA-256 certificate fingerprint** under *App signing key certificate* — the long `AB:CD:EF:...` string. **Not** the upload key certificate. Getting these two mixed up is the usual cause of the app opening with a browser address bar showing.
6. Send me that fingerprint, or paste it yourself into `.well-known/assetlinks.json` replacing `REPLACE_WITH_SHA256_FROM_PLAY_CONSOLE_APP_SIGNING`.
7. Deploy the site.
8. Check it is live:

```
curl https://www.turnsomedayintodayone.com/.well-known/assetlinks.json
```

You should see your fingerprint and `application/json`. Until this is right, the app opens with browser chrome around it.

STEP 6 — Store listing

Open the files in `store-listing/` and paste them in. **They are drafts — edit them into your voice before submitting.**

Field	File	Limit
App name	`01-title-and-short-description.md`	30 chars
Short description	`01-title-and-short-description.md`	80 chars
Full description	`02-full-description.md`	4000 chars

Do not move or reword the first paragraph of the full description. It is the medical-device disclaimer and it is required.

Privacy policy URL — paste exactly:

```
https://www.turnsomedayintodayone.com/privacy
```

Graphics you must supply

Asset	Size	How many	Required?
App icon	512 × 512 PNG, 32-bit	1	Yes

Feature graphic	1024 × 500 PNG or JPG	1	Yes
Phone screenshots	16:9 or 9:16, each side 320–3840 px	**2 minimum**, 8 max	Yes
7-inch tablet	same rules	up to 8	No
10-inch tablet	same rules	up to 8	No

You have `icons/icon-512.png` already. The feature graphic and screenshots you do not have — take the screenshots on a real phone once the app installs.

Screenshot rule that gets apps rejected in this category: no text overlay claiming an outcome. "Day 30" is fine. "Beat your addiction" is not.

STEP 7 — App content declarations

Play Console → **App content**. Work through every card:

1. **Privacy policy** — the URL above.
2. **Ads** — *No, my app does not contain ads.*
3. **App access** — *All functionality is available without special access.* (Anyone can create a free account.)
4. **Content rating** — fill in the questionnaire. Free. Answer honestly; expect a **Teen / PEGI 12** style rating because of mature themes.
5. **Target audience** — **18 and over.** Do not include under-18 age bands. It pulls in the Families policy and a stack of extra requirements you do not want.
6. **Data safety** — answers are in `store-listing/03-data-safety-answers.md`.
7. **Health apps declaration** — answers are in `store-listing/04-health-declaration.md`. **Read point 4 in that file before you submit it.**
8. **Government apps** — No.
9. **Financial features** — No.

STEP 8 — The 12 testers / 14 days requirement

Because your developer account is personal and was created after 13 November 2023, you cannot publish to production until you have run a closed test with **at least 12 testers opted in, continuously, for 14 days.**

1. **Testing** → **Closed testing** → **Create a track** (the default *Alpha* track is fine).
2. **Testers** tab → create an email list → add **at least 12 Gmail addresses**.
   - They must be real people on real Android devices with real Google accounts.
   - Emulators, duplicate accounts and bots do not count.
   - Add 14–15, not exactly 12, so one drop-out does not reset you.
3. Copy the **opt-in URL** and send it to them.
4. **Each tester must click the link, accept, and actually install the app.** An invite that is never accepted does not count. This is where it usually fails.
5. Leave it running **14 consecutive days.** Do not remove testers or pause the track — the counter resets.
6. After 14 days: **Production** → **Apply for production access.** A three-part form. Google usually answers within 7 days.

Who to ask: anyone with an Android phone. They do not have to use the app seriously; they have to install it and stay opted in. Recovery-community contacts, family, friends. Ask 20 to get 12.

STEP 9 — Verify the wrapper before you ship it

Once the app installs on a real device:

1. Open it. **There must be no browser address bar.** If there is, assetlinks is wrong — recheck STEP 5, and confirm you copied the *app signing* fingerprint rather than the upload key one.
2. You should land on the app, logged out, at the sign-in screen.
3. Create an account and complete onboarding.
4. **Confirm there is no way to pay anywhere in the app** — no prices, no "Upgrade to Pro", no Plans screen, no link out to a payment page. This is the single thing most likely to get the app rejected. Check: Home header chip, Profile, the Nova daily-limit message, and the in-app guide search for "price".
5. Confirm an existing Pro account created on the web still shows Pro features.

Emulator (optional, on your machine)

If you want to test without a phone, install Android Studio (free), then:

```
sdkmanager "system-images;android-36;google_apis_playstore;x86_64"
avdmanager create avd -n dayone -k "system-images;android-36;google_apis_playstore;x86_64"
emulator -avd dayone
adb install -r app-release-bundle.aab
```

Note the emulator must be a **Google Play** image, not plain AOSP, or the TWA will not verify.

STEP 10 — Submit

Once closed testing has run its 14 days and production access is granted: **Production** → **Create new release** → upload the same `.aab` → roll out.

If it gets rejected

Rejections in this category are almost always one of four things:

1. **A payment surface was found in the app.** Re-check STEP 9 point 4.
2. **The medical disclaimer is missing from the first paragraph.**
3. **The data safety form does not match the privacy policy.** They are read together.
4. **An outcome claim** in the description or a screenshot.

Send me the rejection text and I will tell you which one it is.